

## 100 Days of Azure DevOps

## Day 68 Handout — Build → Security → Deploy, with dependsOn — a parallel 'security job' is a race, not a gate

7 - Security, Compliance & Governance · Personal lab only · Educational content · Not a sales pitch

### Architecture A — stage order is a control plane

| Stage           | Job                                                              | Gate                                                       |
|-----------------|------------------------------------------------------------------|------------------------------------------------------------|
| Build           | Compile / Docker@2 push to ACR                                   | Must succeed before scans that need the image/artifact     |
| Security        | Day 67 scans + any image scan. Exit non-zero on written criteria | dependsOn: Build. Deploy must not start on failure         |
| Deploy          | ACI/AKS/App Service                                              | dependsOn: Security. condition: succeeded()                |
| PR validation   | Same Security jobs on pull requests when you can                 | Main-only scans are how badness waits for merge            |
| continueOnError | A confession on the Security job                                 | If present, you have a dashboard of regret, not shift-left |

### Architecture B — write fail criteria so the release date cannot vote

| Criterion                          | Default                           | Exception you will actually write                                   |
|------------------------------------|-----------------------------------|---------------------------------------------------------------------|
| Secret detected                    | Fail Security. Deploy stays still | None                                                                |
| Critical CVE in shipped image/deps | Fail                              | Time-boxed waiver in markdown with an expiry — not a silent skip    |
| High CVE                           | Fail if the tool distinguishes    | Warn week one, fail week two — dated                                |
| Scanner outage                     | Fail closed (job fails) or rerun  | Fail open only with a named owner. Lab: fail closed                 |
| Where it runs                      | PR + main                         | Main-only because PRs 'are slower' — that is the final-boss pattern |

### Architecture C — pitfalls that still ship while Security is 'running'

| Pitfall                             | What you see                             | Fix                                   |
|-------------------------------------|------------------------------------------|---------------------------------------|
| Security // Deploy parallel         | Deploy green, scan red two minutes later | dependsOn: Security on Deploy         |
| condition: always() on Deploy       | Deploy runs on failed scans              | condition: succeeded()                |
| Stage named Security, job empty     | Theatre                                  | Put the real scan steps in that stage |
| continueOnError on the gate         | Badge is yellow, prod changed            | Delete the flag                       |
| Weekly security meeting as the gate | Release date always wins                 | Non-zero exit next to the commit      |

## One-liner to remember

Security as a final boss stage is how you ship late — put checks next to the commit. If Deploy still runs when Security is red, the dependsOn is theatre.

## Step-by-step lab (personal Azure / Azure DevOps only)

Personal azure-100-labs. Reuse Day 67 steps inside a Security stage. Prove Deploy does not start on a forced failure.

- 1 Reshape the pipeline into stages: Build, Security, Deploy. Security dependsOn Build. Deploy dependsOn Security with condition succeeded().
- 2 Move secret scan / SCA into Security. Keep fail-closed on secrets.
- 3 Write docs/security-gates-day68.md: fail criteria (secret=fail, critical=fail, high=...). Two sentences, not a novel.
- 4 Force a failure once (exit 1 in a throwaway step, or a fake finding). Confirm Deploy is skipped. If Deploy still runs, fix dependsOn/condition before anything else.
- 5 If PR pipelines exist, add the Security job there. Main-only is a note in the doc as a gap, not a boast.
- 6 Do not add continueOnError to look green on LinkedIn. Restore the pipeline to a passing dummy after the forced-fail proof.

## Starter YAML — stages with a real dependsOn

```
stages:
- stage: Build
  jobs:
  - job: compile
    pool: { vmImage: ubuntu-latest }
    steps:
    - script: echo build
- stage: Security
  dependsOn: Build
  jobs:
  - job: gates
    pool: { vmImage: ubuntu-latest }
    steps:
    - script: echo "secret scan + SCA live here"
    - script: exit 0    # flip to 1 to prove Deploy does not start
      displayName: Forced-fail switch
- stage: Deploy
  dependsOn: Security
  condition: succeeded()
  jobs:
  - job: push
    pool: { vmImage: ubuntu-latest }
    steps:
    - script: echo deploy
```

## Done checklist

- Three stages in that order with dependsOn
- Forced failure skipped Deploy
- Fail criteria written in markdown
- continueOnError not on the Security gate

## Tomorrow — Day 69

Secure pipeline design — boring on purpose: approvals, locks, scoped connections